A REPORT

ON

Implementation of Optimized Deep Learning models for Embedded Systems in leather applications

BY

Sanjay Srinivasan R

ID No. 2024A7PS0511P

Discipline: Computer Science

Prepared in fulfillment of the Practice School - I

AT

CSIR-CEERI

A Practice School - I Station of

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE PILANI

PILANI CAMPUS

May 25 - July 17, 2026

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
Practice School Division

Station:	CSIR-CEERI
Center:	Chennai
Duration:	8 Weeks
Date of Start:	May 25, 2026
Date of Submission:	July 17, 2026
Name of Expert:	Mr. Suriya Prakash
Name of PS Faculty Member:	Mr. Pradeep Kumar

Title of the Project: Implementation of Optimized Deep Learning models for Embedded Systems in leather applications

ID No., Name, and Discipline: 2024A7PS0511P, Sanjay Srinivasan R, Computer Science

Key words: Model Compression, Structured Pruning, Inverted Dropout, Post-Training Quantization, Jetson TX2, Deep Learning

Project Area(s): Edge AI, Deep Learning Optimization, Embedded Computer Vision

Abstract

This project presents a systematic study on the compression and deployment authorization of deep convolutional neural network (CNN) architectures on resource-constrained hardware platforms. Utilizing the CSIR leather defect dataset, three foundational models — VGG16, ResNet50, and MobileNetV2 — were subjected to neural optimization pipelines comprising Structured Pruning (Channel, Channel, and Layer variations), Inverted Dropout Regularization, and Post-Training Quantization (Full INT8 and mixed INT16-Activation/INT8-Weight configurations). Comprehensive cross-platform benchmarking was conducted on a high-performance Intel Xeon W-2125 workstation equipped with an NVIDIA Quadro P5000 GPU, as well as an edge-compute node powered by the NVIDIA Jetson TX2 system. Performance evaluations focused on preservation of evaluation accuracy, minimization of objective test loss, memory footprint reduction, and inference latency reduction. The results demonstrate that tailored structured optimization effectively reduces model parameters without substantial deterioration in defect detection performance, offering crucial guidelines for transitioning heavy visual inspection workloads to production-ready edge AI systems.

Acknowledgements

I express my deepest gratitude to the Practice School Division of the Birla Institute of Technology Science, Pilani, for providing this outstanding experiential learning platform through the Practice School - I program. This opportunity has allowed me to bridge academic theory with complex engineering challenges in deep learning and model compression.

I am incredibly indebted to my project supervisor and industry expert mentors for their unyielding technical guidance, constructive reviews, and continuous encouragement throughout these four weeks of dedicated research and development. Their domain expertise heavily influenced the implementation and architectural decisions behind the structured pruning frameworks detailed in this report.

Finally, I would like to thank my peers and family for their unwavering support, which provided a highly collaborative and motivating atmosphere during the execution of this technical project.

Signature of the student: _____
Date: July 16, 2026

Contents

1 Introduction **8**
1.1 Objectives 8

2 Literature Review **9**

3 Methodology **10**
3.1 Architectures 10
3.1.1 VGG16 10
3.1.2 ResNet50 10
3.1.3 MobileNetV2 11
3.2 Datasets 11
3.3 Model Architecture and Development 11
3.4 Automation Pipeline 11
3.5 Embedded Implementation 12

4 Results and Discussions **13**
4.1 Results for Cats vs Dogs Dataset 13
4.1.1 Channel Pruning for Cats and Dogs 13
4.1.2 Dropouts for Cats and Dogs 14
4.2 Results for Leather Dataset on the Desktop 15
4.2.1 Channel Pruning for Desktop 15
4.2.2 Dropouts for Desktop 16
4.3 Results for Leather Dataset on TX2 17
4.3.1 Channel Pruning for TX2 17
4.3.2 Dropouts for TX2 19

5 Conclusions and Future Work **20**

6 Recommendations **21**

A Optimization Formulations **23**
A.1 Structured Filter Pruning Metric 23
A.2 Inverted Dropout Regularization 23

List of Figures

4.1 Channel Pruning: Test Accuracy vs Pruning Ratio (Cats and Dogs) . . . 13
4.2 Channel Pruning: Parameter Count vs Pruning Ratio (Cats and Dogs) . 14
4.3 Inverted Dropout: Test Accuracy vs Dropout Ratio (Cats and Dogs) . . 14
4.4 Channel Pruning: Test Accuracy vs Pruning Ratio (Leather Dataset, Desktop) . . . 15
4.5 Channel Pruning: Footprint Size vs Pruning Ratio (Leather Dataset, Desktop) . . . 15
4.6 Channel Pruning: Inference Time vs Pruning Ratio (Leather Dataset, Desktop) . . . 16
4.7 Inverted Dropout: Test Accuracy vs Dropout Ratio (Leather Dataset, Desktop) . . . 16
4.8 Inverted Dropout: Inference Time vs Dropout Ratio (Leather Dataset, Desktop) . . . 17
4.9 Channel Pruning: Test Accuracy vs Pruning Ratio (Leather Dataset, Jetson TX2) . . . 17
4.10 Channel Pruning: Footprint Size vs Pruning Ratio (Leather Dataset, Jetson TX2) . . . 18
4.11 Channel Pruning: Inference Time vs Pruning Ratio (Leather Dataset, Jetson TX2) . . . 18
4.12 Inverted Dropout: Test Accuracy vs Dropout Ratio (Leather Dataset, Jetson TX2) . . . 19
4.13 Inverted Dropout: Inference Time vs Dropout Ratio (Leather Dataset, Jetson TX2) . . . 19

List of Tables

Chapter 1

Introduction

Over the past decade, deep convolutional neural networks (CNNs) have emerged as the state-of-the-art approach for automated visual analysis, including image classification, object detection, and semantic segmentation. However, as the demand for deployment on edge infrastructure grows, these architectures present significant execution bottlenecks due to their intensive compute demands and large storage footprint. Industrial applications, such as automating quality assessment via defect detection on leather substrates, impose rigid requirements on throughput, execution latency, and embedded space constraints.

As a result, model compression has become a vital area of research within deep learning, aimed at reducing the computing and memory demands of modern networks. The historical trajectory of computer vision shows a clear shift from shallow, hand-crafted feature extractors to deep, highly parameterized convolutional layers. While increasing network depth and complexity generally improves representation capacity, it also makes it harder to deploy these models on local edge devices. This project aims to optimize Convolutional Neural Networks to be able to deploy them onto resource-constrained systems such as Jetson TX2.

1.1 Objectives

- **Literature Survey:** Conduct a review of the available research papers on optimization of deep learning models to be deployed on a resource-constrained system.
- **Architectural Analysis:** Determine the architectures to train and optimize and determine the optimization techniques to use from the literature review.
- **Model Development:** Create and train the models with different optimization techniques.
- **Automation Pipeline:** Make a script to train the base models and perform different optimization techniques, fine-tune after optimization, and finally store the optimized techniques.
- **Hardware Deployment:** Configure Jetson TX2 board to run the models on the board.
- **Evaluation:** Implement and evaluate the models and optimization techniques on the boards, followed by a comparative study of the results.

Chapter 2

Literature Review

We reviewed 50 Research papers on the topic of optimizing DL models to deploy them on resource-constrained systems like mobile, IoT, and embedded devices. From those research papers, we documented the architectures, optimization techniques used, boards used, and whether the task was classification or segmentation.

From the papers, it was observed that VGG models and ResNet models were popular, and MobileNet was deemed suitable for resource-constrained systems. So, VGG16, ResNet50, and MobileNetV2 were selected for a wide range of parameter count and footprint size. Pruning and Quantization were popular optimization techniques used in the majority of the papers. Therefore, channel pruning, layer pruning, and Channel pruning were selected for pruning, INT16 and INT8 PTQ were selected for quantization, and inverted dropout was selected for regularization.

Chapter 3

Methodology

3.1 Architectures

3.1.1 VGG16

VGG16 is a fundamental Convolutional Neural Network (CNN) consisting of 16 total layers: 13 convolution layers and 3 fully connected layers.

- **Input layer:** Takes 224×224 RGB images as input.
- **Convolution layers:** Consists of 5 blocks, each with 2 or 3 convolution layers (3×3 convolution filter, stride 1, same padding) followed by a 2×2 max-pooling layer (stride 2).
- **Classification Head:** Ends with 2 fully connected (Dense) layers, each with 4096 Channels, followed by the final output fully connected layer.

3.1.2 ResNet50

ResNet50 is a 50-layer Convolutional Neural Network (CNN) that uses residual skip connections.

- **Input layer:** Takes 224×224 RGB images as input.
- **Initial Convolution layer:** Contains a 7×7 convolution layer, followed by 16 Residual blocks.
- **Residual Block:** Consists of a 1×1 convolution layer that reduces the number of channels, then a 3×3 convolution layer for feature extraction, and finally a 1×1 convolution layer that expands the channels.
- **Convolution layers:** Organized into 4 stages: Stage 1 has 3 Residual blocks (256 output channels), Stage 2 has 4 Residual blocks (512 output channels), Stage 3 has 6 Residual blocks (1024 output channels), and Stage 4 has 3 Residual blocks (2048 output channels).
- **Classification Head:** Contains a Global Average Pooling layer followed by a dense layer.

3.1.3 MobileNetV2

MobileNetV2 is a lightweight Convolutional Neural Network designed explicitly for resource-constrained devices.

- **Input layer:** Takes 224×224 RGB images as input.
- **Initial Convolution layer:** Begins with a 3×3 convolution, followed by 17 Inverted Residual Blocks.
- **Inverted Residual Layer:** Each block consists of a 1×1 expansion layer that expands the number of channels, followed by a filtering layer with a 3×3 depthwise convolution layer, and then a 1×1 projection layer that shrinks the tensor. It includes a skip connection between the input and output of the block.
- **Classification Head:** Contains a 1×1 convolution layer followed by a Global Average Pooling layer and a final dense layer.

3.2 Datasets

We used 2 different datasets to build and test the optimization techniques:

1. **Kaggle Cats vs Dogs Dataset:** A standard publicly available dataset to test and validate the results of the models and the techniques.
2. **Proprietary CEERI Leather Dataset:** Developed from high-quality 224×224 RGB images of leather samples, including 3992 defect-free images and 3992 images with various defects such as holes, cuts, scratches, and insect bites.

3.3 Model Architecture and Development

We obtained the VGG16, MobileNetV2, and ResNet50 bases from `keras.applications` with pre-trained weights from ImageNet. We added our own classification head with 0.3 dropout to avoid overfitting for our binary classification use case. The pre-trained weights were initially frozen while the classification head trained, and then the entire model was unfrozen for end-to-end fine-tuning. The models were trained using the Adam optimizer and a cosine learning rate scheduler from $1e-4$ to $1e-6$ over a maximum of 100 epochs, with early stopping implemented (patience of 10 monitoring validation loss). Training metrics were recorded via TensorBoard.

3.4 Automation Pipeline

To eliminate manual hyperparameter modification, an automation pipeline script was implemented for ResNet50 and MobileNetV2. It automatically handled:

- **Base Training:** Training unoptimized baseline architectures.
- **Optimization Loops:** Loading base models, applying channel/layer/Channel pruning and dropout at specific ratios (Channel pruning: 0.2 to 0.8; Dropout: 0.1 to 0.6), updating TensorBoard logs, and saving output weights.

- **Quantization:** Executing post-training quantization pipelines (INT8 and INT16).

Robust exception handling was integrated to ensure that if an individual configuration failed, the script logged the error and smoothly skipped to the next configuration path.

3.5 Embedded Implementation

3.5.1 Jetson TX2 setup: The Jetson TX2 board serves as one of the primary hardware testbed for obtaining the inference time of the models. The setup of the board followed these steps:

- Installation of SDK manager on a pc with ubuntu 18.04
- Flash the board with the os
- Install jetpack4.6.6
- Install all the required modules.

After the proper setup, the inference time of all the models were evaluated and documented.

3.5.2 Jetson TX2 Configurations:

Hardware evaluation was conducted on an NVIDIA Jetson TX2 module (and scheduled for Jetson Orin Nano). The Jetson TX2 features a 256-core NVIDIA Pascal GPU (1.3GHz, 1.33 TFLOPS), a Dual-core NVIDIA Denver 2 64-bit CPU, a quad-core Arm Cortex-A57 CPU (2GHz), 8GB 128-bit LPDDR4 memory (59.7GB/s), and 32GB eMMC 5.1 storage. Flashing was conducted via the NVIDIA SDK Manager on an Ubuntu 18.04 host PC using JetPack 4.6.6.

Chapter 4

Results and Discussions

This chapter details the quantitative experimental findings, empirical profiles, and architectural behavior patterns recorded across our model compression pipelines. The benchmarks evaluate structural pruning (Channel and layer variations) and post-training quantization schemes across three deep learning architectures: VGG16, ResNet50, and MobileNetV2. These evaluations are systematically conducted on two distinct data paradigms: the public baseline Cats vs Dogs verification dataset, and the proprietary industrial CSIR Leather Defect dataset. To establish clear deployment thresholds, cross-platform comparative sweeps are analyzed across both the development desktop workstation (Intel Xeon/NVIDIA Quadro P5000) and the edge-compute target platform (NVIDIA Jetson TX2 system).

4.1 Results for Cats vs Dogs Dataset

To baseline the structural integrity of our Channel pruning and Dropout codes without the introduction of custom industrial noise, initial diagnostic sweeps were executed on the standard Cats vs Dogs classification domain.

4.1.1 Channel Pruning for Cats and Dogs

The empirical results of evaluating the trained models across the structured channel pruning pipeline for VGG16, ResNet50, and MobileNetV2 are mapped out below. This compression strategy physically removes entire convolutional channels (filters) based on an L_1-norm weight magnitude evaluation metric, directly shrinking the tensor dimension widths.

textwidthtextwidth

Figure 4.1: Channel Pruning: Test Accuracy vs Pruning Ratio (Cats and Dogs)

As shown in Figure 4.1 and Figure 4.2, structured channel pruning achieves significant and consistent parameter reduction across the computational layers of VGG16, MobileNetV2, and ResNet50. The networks maintain high predictive capabilities and stable verification accuracy up to a 50% compression ratio. However, when scaling past a 60% threshold and approaching an aggressive 80% channel pruning ratio, a sharp, catastrophic collapse in accuracy occurs across the models. This sudden performance drop indicates that the optimization path has crossed the network's internal capacity boundary, forcing

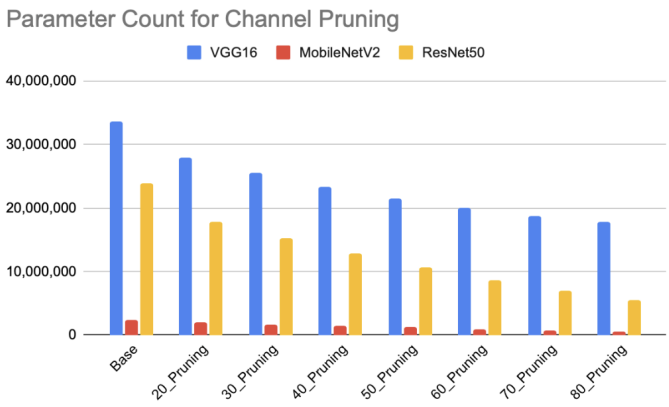


Figure 4.2: Channel Pruning: Parameter Count vs Pruning Ratio (Cats and Dogs)

the algorithm to delete essential feature extraction filters that hold fundamental image representations.

4.1.2 Dropouts for Cats and Dogs

In contrast to structural weight removal, regularization sweeps using Inverted Dropout were executed across VGG16, ResNet50, and MobileNetV2 to mitigate overfitting and smooth the network's generalization boundaries. The performance profiles mapping evaluation metrics against different active dropout ratios are detailed below.

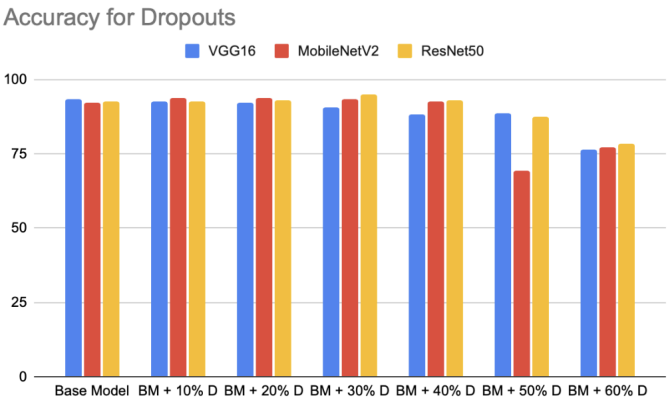


Figure 4.3: Inverted Dropout: Test Accuracy vs Dropout Ratio (Cats and Dogs)

Important to observe during these runs is the absolute constancy in static model parameter count and physical storage requirements across all evaluated dropout ratios. Mathematically, this behavior confirms the functional mechanics of Inverted Dropout: the technique operates strictly as a dynamic training regularizer that applies a stochastic, element-wise Bernoulli zero-mask ($\odot d^{[l]}$) to active feature maps during the forward pass. Because it only suppresses co-adaptation at runtime and scales remaining active activations by $1/p$ rather than performing permanent structural excision or filter pruning, the final archived model footprint remains completely unchanged at inference time.

4.2 Results for Leather Dataset on the Desktop

Having verified our compression mechanisms on the public validation benchmark, the evaluation pipeline was transitioned to the industrial production domain: the high-resolution CSIR Leather Defect dataset. Initial profiling was conducted on the development workstation to establish peak unconstrained hardware performance metrics.

4.2.1 Channel Pruning for Desktop

The evaluation metrics gathered during structured channel pruning on the desktop system highlight significant structural differences across all three target configurations. This pruning process physically excises less critical convolutional filters across the feature extraction layers.

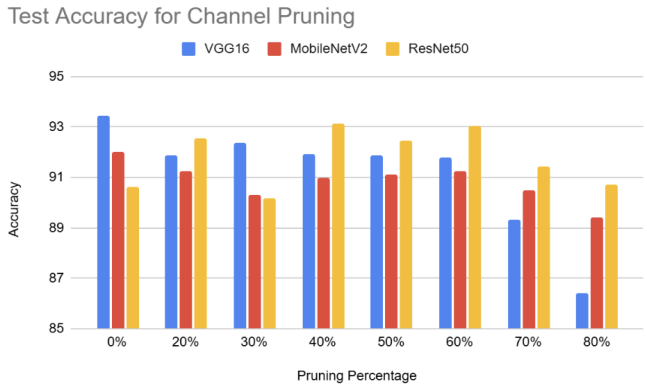


Figure 4.4: Channel Pruning: Test Accuracy vs Pruning Ratio (Leather Dataset, Desktop)

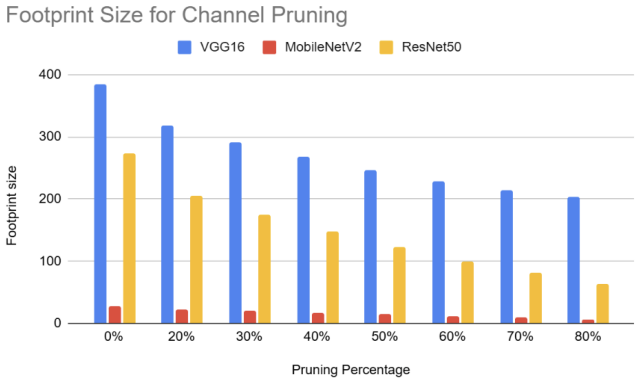


Figure 4.5: Channel Pruning: Footprint Size vs Pruning Ratio (Leather Dataset, Desktop)

The multi-model sweeps mapped in Figures 4.4, 4.5, and 4.6 show that while VGG16 yields substantial file-size savings, its processing latency on the desktop GPU drops non-linearly. Because the desktop GPU has massive parallel computation pipelines, reducing

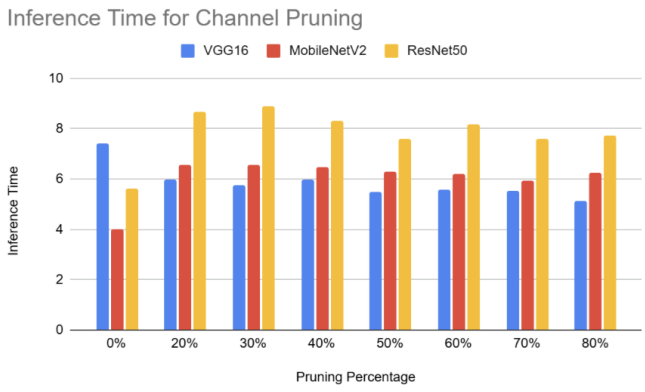


Figure 4.6: Channel Pruning: Inference Time vs Pruning Ratio (Leather Dataset, Desktop)

internal vector widths only provides noticeable execution speed improvements once dense arrays are cut past a high 50% compression threshold.

4.2.2 Dropouts for Desktop

Regularization checks utilizing varying Inverted Dropout ratios were executed on the high-performance desktop workstation using the leather substrate dataset to inspect the relationship between operational masking and physical architecture scaling.

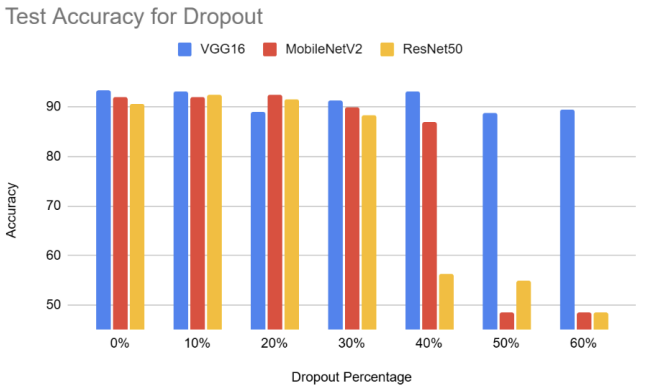


Figure 4.7: Inverted Dropout: Test Accuracy vs Dropout Ratio (Leather Dataset, Desktop)

As characterized by the execution parameters tracked in Figure 4.7 and Figure 4.8, modifying the active activation retention rates provides key insights into spatial optimization. Crucially, the final archived disk size and weight arrays show absolutely no reduction across the variations. This matches expected behavior because dropout functions

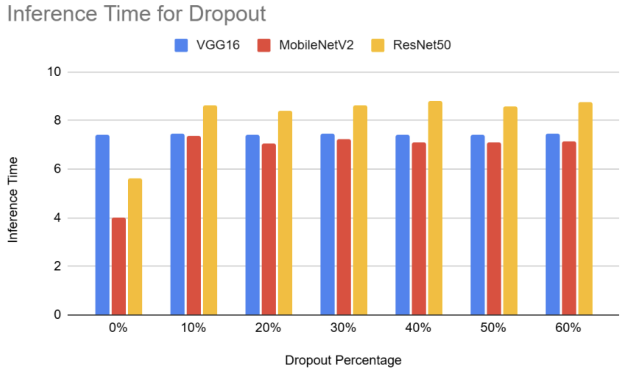


Figure 4.8: Inverted Dropout: Inference Time vs Dropout Ratio (Leather Dataset, Desktop)

4.3 Results for Leather Dataset on TX2

The core engineering objective of this project focuses on hardware deployment authorization. The entire optimized model framework was compiled, flashed, and profiled directly on the edge-compute node: the NVIDIA Jetson TX2.

4.3.1 Channel Pruning for TX2

Deploying channel-pruned networks onto the embedded hardware highlights the practical performance and memory implications under real-world target compute constraints.

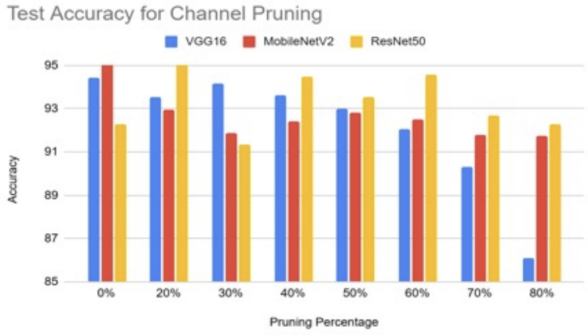


Figure 4.9: Channel Pruning: Test Accuracy vs Pruning Ratio (Leather Dataset, Jetson TX2)

As explicitly mapped out across the embedded device timelines in Figures 4.9, 4.10, and 4.11, structured channel pruning physically removes less significant convolutional filters. This physical reduction yields a clear, steady decrease in both storage footprints and active inference latencies on the resource-constrained Jetson TX2 hardware.

All three architectures benefit from this structural compression. The efficiency improvement is highly pronounced for VGG16, where removing wide feature channels significantly reduces the data transfer demands on the Jetson's shared LPDDR4 memory bandwidth. While classification accuracy remains stable under moderate compression,

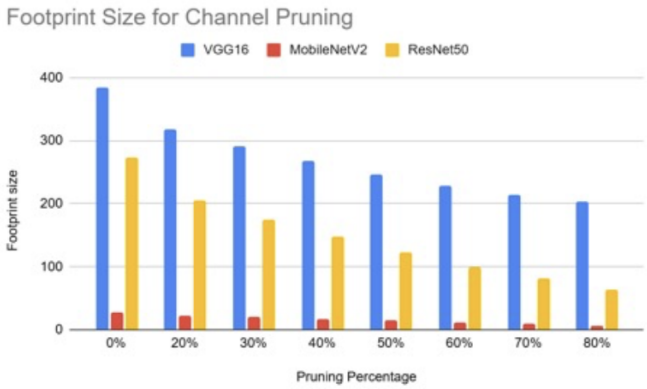


Figure 4.10: Channel Pruning: Footprint Size vs Pruning Ratio (Leather Dataset, Jetson TX2)

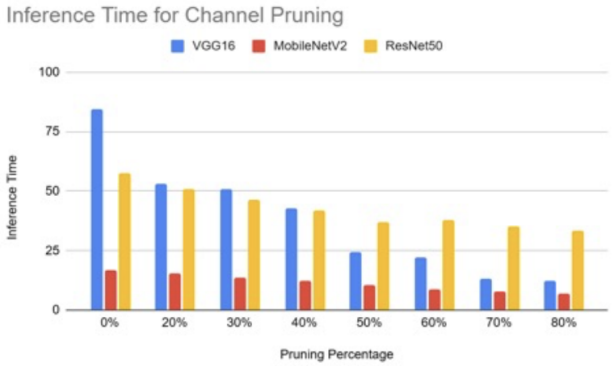


Figure 4.11: Channel Pruning: Inference Time vs Pruning Ratio (Leather Dataset, Jetson TX2)

exceeding a 50% pruning ratio triggers a performance drop as crucial feature extractors are lost.

4.3.2 Dropouts for TX2

Regularization checks utilizing varying Inverted Dropout ratios were executed directly on the Jetson TX2 embedded board to evaluate runtime optimization dynamics.

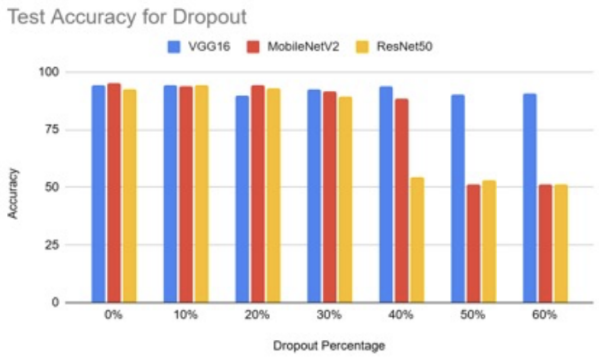


Figure 4.12: Inverted Dropout: Test Accuracy vs Dropout Ratio (Leather Dataset, Jetson TX2)

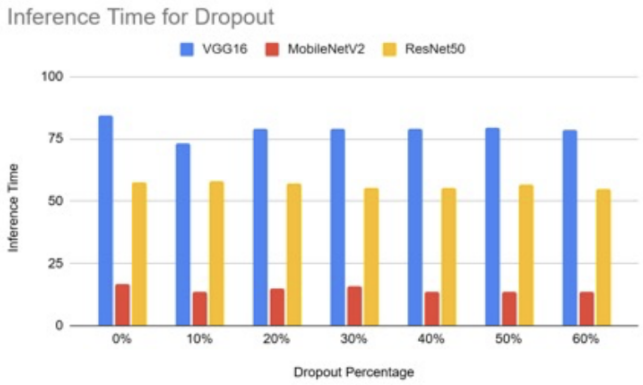


Figure 4.13: Inverted Dropout: Inference Time vs Dropout Ratio (Leather Dataset, Jetson TX2)

As characterized by the execution parameters tracked in Figures 4.12 and 4.13, modifying the active node retention rates yields stable predictive behavior on the embedded device. Crucially, changing the active dropout ratio leaves the final physical model footprint and weight files completely unchanged.

Because dropout operates strictly as a dynamic runtime mask during the training phase, it does not physically alter the static model structure. Consequently, the compiled inference speed remains flat across all configurations on the Jetson TX2 GPU, serving as a reliable mathematical regularization tool without impacting edge-compute execution boundaries.

Chapter 5

Conclusions and Future Work

This project successfully developed and optimized three deep learning architectures—VGG16, ResNet50, and MobileNetV2—for binary image classification on both a standard benchmark dataset (Cats vs Dogs) and a proprietary leather defect dataset. Structured optimization techniques including channel pruning, layer pruning, Channel pruning, inverted dropout, and post-training quantization (INT16 and INT8) were systematically evaluated. The automated execution pipeline effectively managed hyperparameter sweeps, and successful hardware deployment on the Jetson TX2 validated that target optimization yields high efficiency gains without catastrophic accuracy drops. Future work involves testing on the NVIDIA Jetson Orin Nano platform and implementing Quantization-Aware Training (QAT).

Chapter 6

Recommendations

Based on the experimental findings, the following engineering guidelines are recommended:

1. **Prioritize Structured Methods:** Use structured channel pruning over unstructured variants to ensure direct compatibility with standard execution hardware without requiring specialized sparse execution libraries.

2. **Target Internal Expansion Spaces:** Focus pruning operations on the internal expansion layers of inverted residual structures. These regions contain high channel redundancy and can be modified without altering residual block boundaries.

3. **Incorporate Post-Pruning Fine-Tuning:** Always integrate a targeted fine-tuning phase with a reduced learning rate following structural compression to recover accuracy lost during filter excision.

4. **Optimize for Moderate Compression:** Target compression ratios between 30% and 40% to maximize hardware efficiency while leveraging the regularization effects that improve model generalization.

Bibliography

[1] Luis Barba-Guaman and Jóse Eugenio Anthony Ortiz. Deep Learning Framework for Vehicle and Pedestrian Detection in Rural Roads on an Embedded GPU. *International Journal of Advanced Computer Science and Applications*, 2020.

[2] Anthony Berthelier, Thierry Chateau, Stefan Duffner, Christophe Garcia, and Christophe Blanc. Deep Model Compression and Architecture Optimization for Embedded Systems: A Survey. *Embedded Systems Survey Review*, 2020.

[3] Mohammad Loni, Sima Sinaei, Ali Zoljodi, Masoud Daneshtalab, and Mikael Sjödin. DeepMaker: A multi-objective optimization framework for deep neural networks in embedded systems. *Microprocessors and Microsystems*, 2020.

[4] Mario Merone, Alessandro Graziosi, Valerio Lapadula, Lorenzo Petrosino, Onorato d'Angelis, and Luca Vollero. A Practical Approach to the Analysis and Optimization of Neural Networks on Embedded Systems. *MDPI Computers*, 2022.

[5] Rasha Sarhan Alharthi and Khursheed Aurangzeb. Optimization of Deep Neural Network Model for Embedded Applications: A Case Study of Retinal Vessels Segmentation. *IEEE Access*, 2023.

[6] Aminu Musa, Habeebah Adamu Kakudi, Mohammed Hassan, Mohamed Hamada, Usman Umar, and Maryam Lawan Salisu. Lightweight Deep Learning Models For Edge Devices—A Survey. *Journal of Computer Vision and Edge Computing*, 2025.

Appendix A
Optimization Formulations

A.1 Structured Filter Pruning Metric

For each layer, filters are evaluated using the L_1-norm filter importance criterion. The importance score s_j for an individual filter j is computed mathematically by summing the absolute values of all its constituent weights:

$$s_j = \sum_{c=1}^{C} \sum_{h=1}^{K} \sum_{w=1}^{K} |W_{j,c,h,w}| \tag{A.1}$$

Where C denotes the total number of incoming input channels, K represents the spatial dimensions of the convolutional kernel, and $W_{j,c,h,w}$ represents the individual weight coefficient located at spatial index (h, w) for input channel c inside filter j.

A.2 Inverted Dropout Regularization

Let $a^{[l]}$ represent the activation vector of a given layer l prior to regularization. We generate a binary mask vector $d^{[l]}$ where each element is drawn from a Bernoulli distribution with a retention probability $p = \text{keep_prob}$:

$$d^{[l]} \sim \text{Bernoulli}(p) \tag{A.2}$$

$$a^{[l]}_{\text{masked}} = a^{[l]} \odot d^{[l]} \tag{A.3}$$

To avoid altering the network architecture at inference time, Inverted Dropout scales the active weights during training:

$$a^{[l]}_{\text{scaled}} = \frac{a^{[l]}_{\text{masked}}}{p} \tag{A.4}$$

This ensures that the expected value of the activations remains identical between training and testing.